- 训练验证报告
  - 一、训练方案详情
    - 方案 A：YOLOv8n 直接训练（图片压缩到 640×640）
    - 方案 B：YOLOv8n 切片训练（640×640 切片，25% 重叠）
    - 方案 C：YOLOv8m 切片训练
    - 方案 D：YOLO11m 切片训练
  - 二、四方案核心对比
    - 整体指标
    - 分类别指标
      - 异物检测
      - 小NC过切检测
  - 三、关键发现
    - 1. 切片对微小缺陷至关重要
    - 2. 大模型不一定更好
    - 3. YOLOv8m 在异物精度上略有优势
    - 4. YOLO11m 整体均衡
    - 5. 小NC过切在所有切片方案中都有所下降
  - 四、模型文件
  - 五、推荐方案
  - 六、下一步优化建议
  - 七、方案 E：YOLOv8n + 切片 + 负样本图片
    - 方案 B vs E 对比（均为 YOLOv8n 切片训练）
  - 八、方案 F：YOLOv8n + 切片 + 负样本图片（200 epoch）
    - 方案 E(100ep) vs F(200ep) 对比


# 训练验证报告

> 生成时间：2026-04-25 硬件环境：NVIDIA GeForce RTX 3080 (10GB) / PyTorch 2.11.0 + CUDA 12.6 数据集：batch1_coco（311 张训练图，88 张验证图，49 张测试图，2 类缺陷）

---

# 一、训练方案详情

## 方案 A：YOLOv8n 直接训练（图片压缩到 640×640）

5472×3648 的高分辨率原图由 Ultralytics 自动缩放到 640×640 进行训练。

**命令：** `python main.py train --dataset dataset/batch1_coco --model yolov8n.pt --epochs 100 --batch-size 16 --device cuda --no-slice`

**训练参数：**

| 项目 | 值 |
|---|---|
| 模型 | YOLOv8n（301 万参数，8.2 GFLOPs） |
| Epochs | 100 |
| Batch Size | 16 |
| 图片尺寸 | 640（自动缩放） |
| GPU 显存占用 | ~1.9 GB |
| 总耗时 | 0.124 小时（约 7.5 分钟） |
| 每 Epoch 耗时 | ~2.7 秒 |

**最终验证结果（best.pt）：**

| 类别 | Precision | Recall | mAP50 | mAP50-95 |
|---|---|---|---|---|
| **整体** | 0.565 | 0.518 | 0.511 | 0.371 |
| 异物 | 0.415 | 0.143 | 0.110 | 0.028 |
| 小NC过切 | 0.715 | 0.892 | 0.912 | 0.715 |

**推理速度：** 1.4ms / 张

---

# 方案 B：YOLOv8n 切片训练（640×640 切片，25% 重叠）

5472×3648 的原图被切成 640×640 的小片（重叠率 25%），每张图约产生 77 个切片。含缺陷的切片保留，不含缺陷的按 1:5 比例随机抽样作为负样本。

**命令：** `python main.py train --dataset dataset/batch1_coco --model yolov8n.pt --epochs 100 --batch-size 16 --device cuda`

**切片参数：**

| 项目 | 值 |
| --- | --- |
| 切片大小 | 640×640 |
| 重叠率 | 25% |
| 最小面积比 | 0.3 |
| 负样本比例 | 0.2 |
| 训练切片数 | 721 张 |
| 验证切片数 | 209 张 |
| 测试切片数 | 110 张 |

**训练参数：**

| 项目 | 值 |
| --- | --- |
| 模型 | YOLOv8n（301 万参数，8.2 GFLOPs） |
| Epochs | 100 |
| Batch Size | 16 |
| GPU 显存占用 | ~2.3 GB |
| 总耗时 | 0.394 小时（约 24 分钟） |
| 每 Epoch 耗时 | ~12 秒 |

**最终验证结果（best.pt）：**

| 类别 | Precision | Recall | mAP50 | mAP50-95 |
| --- | --- | --- | --- | --- |
| **整体** | 0.719 | 0.646 | 0.680 | 0.389 |
| 异物 | 0.643 | 0.488 | 0.540 | 0.206 |
| 小NC过切 | 0.796 | 0.805 | 0.819 | 0.572 |

**推理速度：** 1.4ms / 切片

---

# 方案 C：YOLOv8m 切片训练

使用 YOLOv8 的 medium 版本，参数量约为 nano 版的 8.6 倍（2586 万参数），在相同切片数据上训练。

**命令**：`python main.py train --dataset dataset/batch1_coco --model yolov8m.pt --epochs 100 --batch-size 16 --device cuda`

**训练参数**：

| 项目 | 值 |
|---|---|
| 模型 | YOLOv8m（2586 万参数，79.1 GFLOPs） |
| Epochs | 100 |
| Batch Size | 16 |
| GPU 显存占用 | ~6.0 GB |
| 总耗时 | 0.592 小时（约 35 分钟） |
| 每 Epoch 耗时 | ~18 秒 |
| 迁移学习 | 469/475 项权重已迁移 |

**最终验证结果（best.pt）**：

| 类别 | Precision | Recall | mAP50 | mAP50-95 |
|---|---|---|---|---|
| **整体** | 0.772 | 0.569 | 0.621 | 0.376 |
| 异物 | 0.782 | 0.393 | 0.477 | 0.227 |
| 小NC过切 | 0.762 | 0.744 | 0.765 | 0.525 |

**推理速度**：3.7ms / 切片

---

# 方案 D：YOLO11m 切片训练

使用 Ultralytics 最新一代 YOLO11 的 medium 版本（2003 万参数），在相同切片数据上训练。

**命令**：`python main.py train --dataset dataset/batch1_coco --model yolo11m.pt --epochs 100 --batch-size 16 --device cuda`

**训练参数**：

| 项目 | 值 |
|---|---|
| 模型 | YOLO11m（2003 万参数，67.7 GFLOPs） |

| 项目 | 值 |
| --- | --- |
| Epochs | 100 |
| Batch Size | 16 |
| GPU 显存占用 | ~8.1 GB |
| 总耗时 | 0.940 小时（约 56 分钟） |
| 每 Epoch 耗时 | ~25-30 秒 |

**最终验证结果（best.pt）：**

| 类别 | Precision | Recall | mAP50 | mAP50-95 |
| --- | --- | --- | --- | --- |
| **整体** | 0.729 | 0.588 | 0.629 | 0.368 |
| 异物 | 0.661 | 0.417 | 0.461 | 0.184 |
| 小NC过切 | 0.796 | 0.759 | 0.797 | 0.552 |

**推理速度：** 4.7ms / 切片

---

# 二、四方案核心对比

## 整体指标

| 指标 | A: v8n直接 | B: v8n切片 | C: v8m切片 | D: 11m切片 |
| --- | --- | --- | --- | --- |
| **mAP50** | 0.511 | **0.680** | 0.621 | 0.629 |
| **mAP50-95** | 0.371 | 0.389 | 0.376 | **0.398** |
| **Precision** | 0.565 | 0.719 | **0.772** | 0.729 |
| **Recall** | 0.518 | **0.646** | 0.569 | 0.588 |
| 训练耗时 | 7.5 分钟 | 24 分钟 | 35 分钟 | 56 分钟 |
| 推理速度 | 1.4ms | 1.4ms | 3.7ms | 4.7ms |
| 模型大小 | 6.2 MB | 6.3 MB | 52 MB | 40.5 MB |

## 分类别指标

**异物检测**

| 指标 | A: v8n直接 | B: v8n切片 | C: v8m切片 | D: 11m切片 |
|---|---|---|---|---|
| **mAP50** | 0.110 | **0.540** | 0.477 | 0.461 |
| **mAP50-95** | 0.028 | 0.206 | **0.227** | 0.184 |
| **Recall** | 0.143 | **0.488** | 0.393 | 0.417 |

**小NC过切检测**

| 指标 | A: v8n直接 | B: v8n切片 | C: v8m切片 | D: 11m切片 |
|---|---|---|---|---|
| **mAP50** | 0.912 | 0.819 | 0.765 | **0.797** |
| **mAP50-95** | **0.715** | 0.572 | 0.525 | 0.552 |
| **Recall** | **0.892** | 0.805 | 0.744 | 0.759 |

---

# 三、关键发现

## 1. 切片对微小缺陷至关重要

异物的 mAP50 从 0.11（直接训练）飙升到 0.54（切片训练），提升近 4 倍。这证实了切片策略的核心价值——高分辨率图片直接缩到 640×640 时，微小异物的特征被严重压缩甚至丢失，模型几乎学不到任何东西。切片保留了原始分辨率下的细节，让模型能看清这些小目标。

## 2. 大模型不一定更好

出乎意料的是，**YOLOv8n（301 万参数）在切片模式下表现最好**，整体 mAP50 达到 0.680，超过了 YOLOv8m 的 0.621 和 YOLO11m 的 0.629。原因分析：

- 切片后每张图只有 640×640，信息量有限，小模型已经足够拟合
- 大模型在 721 张训练图上更容易过拟合
- YOLOv8n 推理速度最快（1.4ms），是 v8m（3.7ms）的 2.6 倍

## 3. YOLOv8m 在异物精度上略有优势

YOLOv8m 在异物检测上 mAP50-95 达到 0.227（高于 v8n 的 0.206），Precision 也最高（0.782）。说明大模型对边界框定位的精度更好，但召回率较低（0.393 vs v8n 的 0.488）。

## 4. YOLO11m 整体均衡

YOLO11m 在两个类别上表现比较均衡，小NC过切的 mAP50 达到 0.797（切片方案中最高），但异物 mAP50（0.461）不如 YOLOv8n。训练耗时最长（56 分钟），推理速度也最慢（4.7ms）。

## 5. 小NC过切在所有切片方案中都有所下降

直接训练时小NC过切 mAP50=0.912，切片后降到 0.765-0.819。原因是过切缺陷面积较大，切片把它切成碎片后模型看不到完整形态。可以通过提高重叠率（25%→40%）或增大切片尺寸（640→1024）来改善。

---

# 四、模型文件

| 方案 | 模型 | 路径 | 文件大小 |
|---|---|---|---|
| A: v8n 直接 | YOLOv8n | `runs/detect/runs/train_20260425_084657/weights/best.pt` | 6.2 MB |
| B: v8n 切片 | YOLOv8n | `runs/detect/runs/train_20260425_094237/weights/best.pt` | 6.3 MB |
| C: v8m 切片 | YOLOv8m | `runs/detect/runs/train_20260425_131331/weights/best.pt` | 52 MB |
| D: 11m 切片 | YOLO11m | `runs/detect/runs/train_20260425_134949/weights/best.pt` | 40.5 MB |

# 五、推荐方案

**当前推荐方案 B（YOLOv8n + 切片训练）**，理由：

- 整体 mAP50 最高（0.680）
- 异物检测效果最好（mAP50=0.540，Recall=0.488）
- 模型最小（6.3 MB），推理最快（1.4ms）
- 训练时间适中（24 分钟）

**如果更看重定位精度**，可以考虑方案 C（YOLOv8m + 切片），异物的 mAP50-95 最高（0.227），Precision 也最高（0.782）。

---

# 六、下一步优化建议

1. **提高切片重叠率到 40%**：缓解大尺寸缺陷被切断的问题
2. **增大训练 epoch 到 200-300**：切片训练在 100 epoch 时 loss 仍在下降，还有提升空间
3. **增加数据增强**：对异物类别做针对性的 mosaic、mixup 增强，弥补样本量不足
4. **收集更多异物样本**：当前异物标注只有 181 个（小NC过切有 210 个），且形态多变
5. **尝试切片尺寸 1024**：兼顾微小特征保留和大缺陷完整性

---

# 七、方案 E：YOLOv8n + 切片 + 负样本图片

在方案 B 之后，向数据集中补充了大量无缺陷的负样本图片（良品图片），使模型能更好地学习"什么不是缺陷"，减少误检。

**数据集变化：**

| 项目 | 方案 B（之前） | 方案 E（现在） |
|---|---|---|
| 训练图片总数 | 311 | **1064** |
| 正样本（含标注） | 310 (99.7%) | 310 (29.1%) |
| **负样本（无标注）** | 1 (0.3%) | **754 (70.9%)** |
| 验证图片总数 | 88 | **304** |

| 项目 | 方案 B（之前） | 方案 E（现在） |
|---|---|---|
| 正/负样本（验证） | 87/1 | 87/217 |
| 测试图片总数 | 49 | **160** |
| 正/负样本（测试） | 46/3 | 46/114 |

**命令：** `python main.py train --dataset dataset/batch1_coco --model yolov8n.pt --epochs 100 --batch-size 16 --device cuda`

**切片参数：**

| 项目 | 值 |
|---|---|
| 切片大小 | 640×640 |
| 重叠率 | 25% |
| 负样本图片处理 | 每张随机抽取 2 个切片 |
| 训练切片数 | 2168 张（含缺陷切片 + 负样本切片） |
| 验证切片数 | 640 张 |
| 测试切片数 | 335 张 |

**训练参数：**

| 项目 | 值 |
|---|---|
| 模型 | YOLOv8n（301 万参数，8.2 GFLOPs） |
| Epochs | 100 |
| Batch Size | 16 |
| GPU 显存占用 | ~2.3 GB |
| 总耗时 | 约 70 分钟 |
| 每 Epoch 耗时 | ~40 秒 |
| Best Epoch | 69 |

**最终验证结果（best.pt，Epoch 69）：**

| 类别 | Precision | Recall | mAP50 | mAP50-95 |
|---|---|---|---|---|
| **整体** | 0.797 | 0.601 | 0.647 | 0.390 |

**推理速度**：1.4ms / 切片

---

## 方案 B vs E 对比（均为 YOLOv8n 切片训练）

| 指标 | B: 无负样本图片 | E: 含754张负样本图片 | 变化 |
|---|---|---|---|
| **mAP50** | 0.680 | 0.647 | -4.9% |
| **mAP50-95** | 0.389 | **0.390** | +0.3% |
| **Precision** | 0.719 | **0.797** | **+10.8%** |
| **Recall** | **0.646** | 0.601 | -7.0% |
| 训练切片数 | 721 | 2168 | 3x |
| 训练耗时 | 24 分钟 | 70 分钟 | 2.9x |

**分析**：

- **Precision 提升显著**（0.719→0.797，+10.8%）：负样本帮助模型更好地识别"什么不是缺陷"，减少了误检
- **Recall 有所下降**（0.646→0.601，-7.0%）：模型变得更"谨慎"，对一些边界案例不再轻易判定为缺陷
- **mAP50 略降**（0.680→0.647）：Precision 的提升没有完全弥补 Recall 的下降
- **mAP50-95 基本持平**（0.389→0.390）：定位精度没有变化

**结论**：负样本图片在工业场景中是**必要的**。虽然整体 mAP50 略降，但 Precision 提升了 11%，意味着在实际部署中**误报更少**。对于工业检测来说，减少误报（避免把良品判为不良品导致浪费）通常比提高召回率更重要。

**模型文件**：`runs/detect/runs/train_20260425_164524/weights/best.pt`（6.3 MB）

---

# 八、方案 F：YOLOv8n + 切片 + 负样本图片（200 epoch）

与方案 E 完全相同的数据集和配置，仅将 epochs 从 100 提升到 200，观察更长时间训练是否能进一步提升。

**命令：** `python main.py train --dataset dataset/batch1_coco --model yolov8n.pt --epochs 200 --batch-size 16 --device cuda`

**训练参数：**

| 项目 | 值 |
|---|---|
| 模型 | YOLOv8n（301 万参数，8.2 GFLOPs） |
| Epochs | 200（实际执行 106，EarlyStopping 终止） |
| EarlyStopping patience | 50 |
| Best Epoch | **56** |
| 总耗时 | 1.238 小时（约 74 分钟） |
| 每 Epoch 耗时 | ~40 秒 |

**最终验证结果（best.pt，Epoch 56）：**

| 类别 | Precision | Recall | mAP50 | mAP50-95 |
|---|---|---|---|---|
| **整体** | 0.715 | 0.581 | 0.636 | 0.367 |
| 异物 | 0.637 | 0.418 | 0.501 | 0.189 |
| 小NC过切 | 0.793 | 0.744 | 0.772 | 0.545 |

**推理速度：** 1.1ms / 切片

---

# 方案 E(100ep) vs F(200ep) 对比

| 指标 | E: 100 epoch | F: 200 epoch | 变化 |
|---|---|---|---|
| **mAP50** | 0.647 | 0.636 | -1.7% |
| **mAP50-95** | 0.390 | 0.367 | -5.9% |
| **Precision** | **0.797** | 0.715 | -10.3% |
| **Recall** | 0.601 | **0.581** | -3.3% |
| Best Epoch | 69 | 56 | — |
| 实际训练 Epoch | 100 | 106（早停） | — |

**分析：**

**200 epoch 并没有带来更好的结果**。实际上所有指标都略低于 100 epoch 方案。关键原因：

1. **EarlyStopping 在 epoch 56 就找到了 best**，之后 50 个 epoch 没有任何改善，说明模型在 epoch 56 左右就已经收敛到最优状态了
2. **两个方案训练的是不同的切片数据**——方案 E 的负样本切片是随机采样的（每张负样本图抽 2 个切片），每次运行采样结果不同，导致训练数据不同
3. **mAP50 差异仅 0.01**（0.647 vs 0.636），在统计波动范围内

**结论**：对于当前数据集规模（2229 张训练切片），100 epoch 已经足够。增加到 200 epoch 不会带来额外提升。如果需要进一步提升效果，应该从数据质量（更多正样本、更精确的标注）和模型策略（调整重叠率、切片尺寸）入手，而不是单纯增加训练轮次。

**模型文件**：`runs/detect/runs/train_20260425_185112/weights/best.pt`（6.3 MB）